

JavaScript DOM Interaction & Client-Side Storage

1. Selecting DOM elements by id, class, tag (getElementById, getElementsByClassName, getElementsByTagName, querySelector / querySelectorAll)
2. Fetch API + practical login example (POST JSON)
3. localStorage vs sessionStorage – usage, differences, security notes

1. Selecting Elements in the DOM

Methods Comparison :

Method	Returns	Live collection?	Selector flexibility	Performance (approx)	Best modern use case
<code>getElementById("id")</code>	single Element or null	no	only id	fastest	unique identifiers (forms, modals, main containers)
<code>getElementsByClassName("cls")</code>	HTMLCollection (live)	yes	single class name only	very fast	multiple similar elements (buttons, cards)
<code>getElementsByTagName("div")</code>	HTMLCollection (live)	yes	single tag only	fast	all inputs, all links, etc.
<code>querySelector("css")</code>	first Element or null	no	any valid CSS selector	slower	complex / first match (<code>.card > h2, #form [type=email]</code>)
<code>querySelectorAll("css")</code>	static NodeList	no	any valid CSS selector	slower	most flexible – modern default for lists

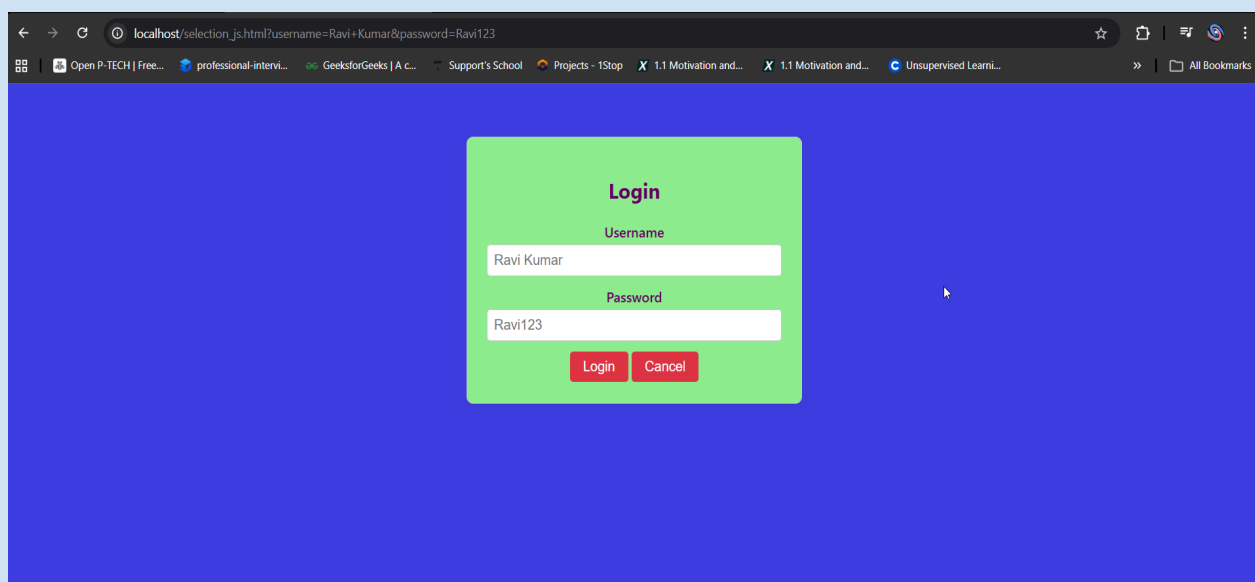
Key differences explained

- **Live vs Static**
 - HTMLCollection (getElementsBy*) updates automatically when DOM changes
 - NodeList from querySelectorAll is snapshot → safer in loops when modifying DOM
- **Modern recommendation (2025)** → Prefer querySelector / querySelectorAll for almost everything → Use getElementById only when you really need max speed on a known ID

The screenshot shows a web browser with a form titled "DOM Selection". The form contains two input fields: "Username" and "Password", and two buttons: "Login" and "Cancel". The form is styled with CSS, including a background color of #f8f9fa, a border of 1px solid #ccc, and a padding of 10px. The buttons are styled with a background color of #6c757d, a color of white, and a font-weight of bold. The form is also styled with a font-family of system-ui, -apple-system, sans-serif, a font-size of 16px, and a line-height of 1.5.

The browser's developer tools are open, showing the HTML and CSS for the form. The HTML shows the form structure, including the input fields and buttons. The CSS shows the styling for the form, including the background color, border, padding, and font settings.

Output:



2. Fetch API – Modern HTTP Client (replaces XMLHttpRequest)

Why Fetch?

- 2015+ → native Promise-based API
- Cleaner syntax than old XMLHttpRequest
- Better error handling, streaming support
- Works in browsers & Node.js (with polyfills or native in recent versions)

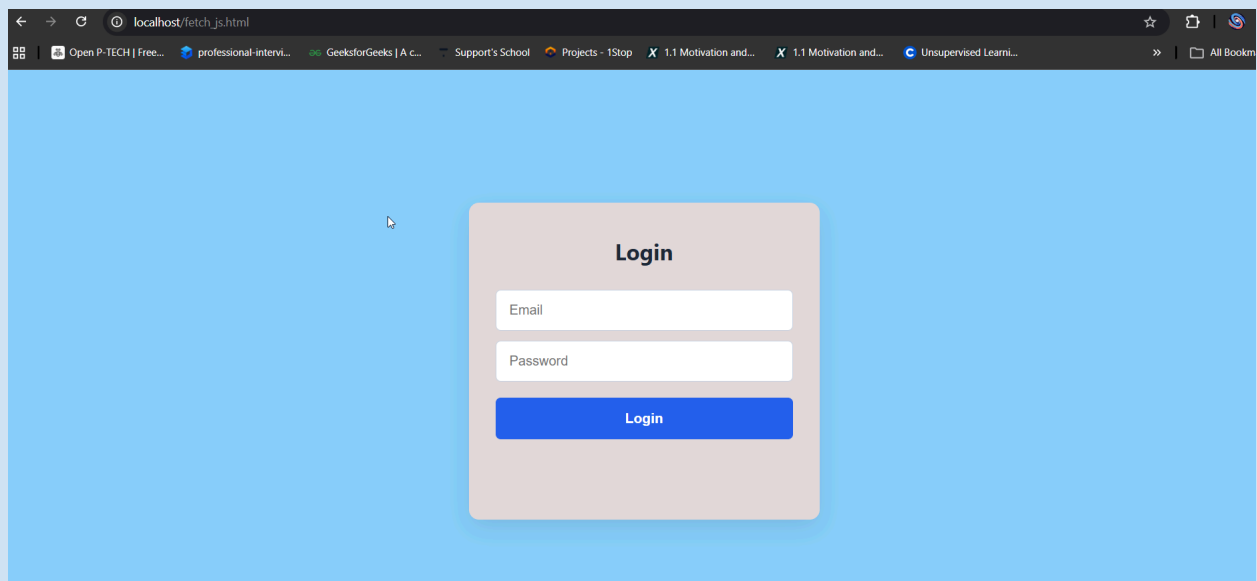
Basic structure

```
fetch(url, {
  method: "POST",           // or GET, PUT, DELETE...
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer token123"
  },
  body: JSON.stringify({ key: "value" })
})
.then(response => {
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();    // .text(), .blob(), etc.
})
.then(data => console.log("Success:", data))
.catch(err => console.error("Fetch failed:", err));
```

Login example with fetch

```
fetch.js.html X
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width">
6   <title>Login Fetch Example</title>
7   <link rel="stylesheet" href="fetch.js.css">
8 </head>
9 <body style="background-color: #lightskyblue;">
10
11   <main>
12     <!--<h2>Login (Client + API simulation)</h2>
13     <h2>Login</h2>
14
15     <form id="login-form" novalidate>
16       <input
17         id="email"
18         type="email"
19         placeholder="Email"
20         required
21         autocomplete="email"
22       >
23
24       <input
25         id="password"
26         type="password"
27         placeholder="Password"
28         required
29         autocomplete="current-password"
30       >
31
32       <button type="submit">Login</button>
33     </form>
34
35     <div id="status" aria-live="polite"></div>
36   </main>
37
fetch.js.css X
1 {
2   box-sizing: border-box;
3
4   body {
5     font-family: system-ui, -apple
6     background: #f8f9fc;
7     margin: 0;
8     min-height: 100vh;
9     display: grid;
10    place-items: center;
11    padding: 1rem;
12  }
13
14  main {
15    width: 100%;
16    max-width: 420px;
17    background: #f8f9fc;
18    padding: 2.5rem 2rem;
19    border-radius: 12px;
20    box-shadow: 0 10px 25px #
21  }
22
23  h2 {
24    margin: 0 0 1.8rem;
25    font-size: 1.65rem;
26    text-align: center;
27    color: #1e293b;
28  }
29
30  input {
31    display: block;
32    width: 100%;
33    padding: 0.9rem 1rem;
34    margin: 0.75rem 0;
35    border: 1px solid #cbd5e1;
36    border-radius: 6px;
37
fetch.js.js X
1 JS fetch.js.js > document.addEventListener(DOMContentLoaded) callback > form
2 document.addEventListener('DOMContentLoaded', () => {
3   form.addEventListener('submit', async (e) => {
4     throw new Error('Login failed HTTP ${response.stat
5   }
6   }
7   const data = await response.json();
8   // In real app: save token to localStorage / httponly
9   // localStorage.setItem('token', data.token);
10  showMessage('Welcome back! (fake token id: ${data.id})
11  } catch (err) {
12    showMessage(
13      err.message.includes('HTTP')
14      ? err.message
15      : 'Network error or server is unreachable',
16      'error'
17    );
18  }
19  });
20
21  function showMessage(message, type) {
22    status.textContent = message;
23    status.className = type;
24  }
25  });
26  });
```

Output:



Notes:

- Always use `async/await` (cleaner than `.then` chains)
- Check `response.ok`
- Handle network errors with `try/catch`
- Never log passwords in production
- Use HTTPS only
- Add timeout with `AbortController` if needed

3. `localStorage` vs `sessionStorage`

Core Comparison Table

Feature	<code>localStorage</code>	<code>sessionStorage</code>	Cookies (for comparison)
Lifetime	forever (until deleted)	until tab/window closed	can be session or persistent
Scope	same origin – all tabs/windows	same origin – current tab only	sent with every request
Size limit	~5–10 MB (browser dependent)	~5–10 MB	~4 KB
Accessible via JS	yes	yes	limited (<code>HttpOnly</code>)
Sent to server	no	no	yes (every HTTP request)
Security risk level	high (persistent XSS can steal forever)	medium (tab-scoped)	medium-high (CSRF, etc.)

Common Use Cases

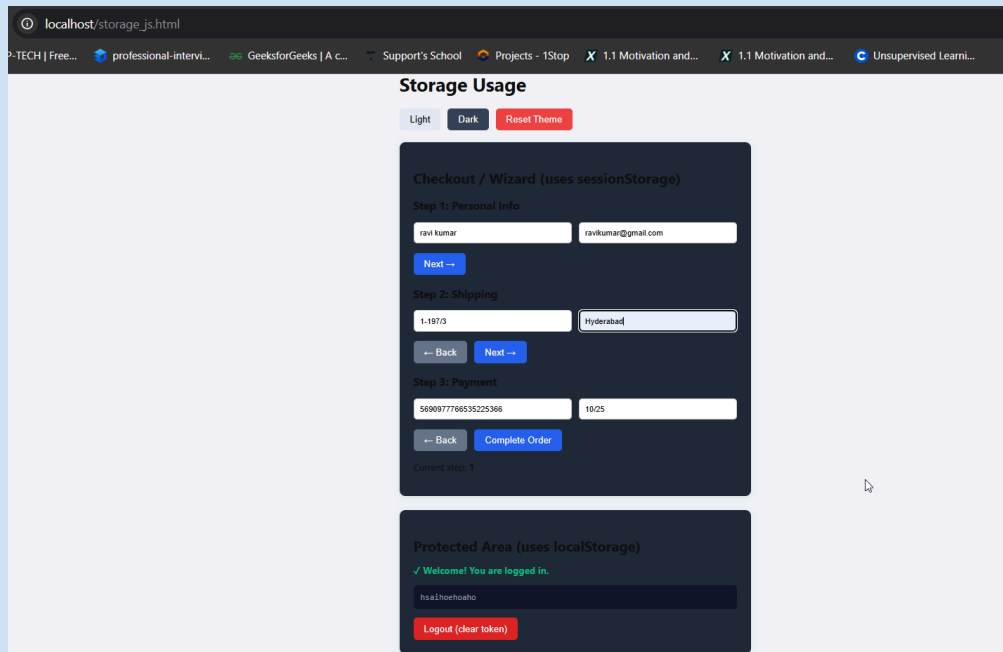
- **localStorage**
 - User preferences/theme (dark mode)
 - Offline cart/wishlist
 - "Remember me" non-sensitive settings
 - Cached API responses (with expiration logic)
- **sessionStorage**
 - Multi-step form data (wizard)
 - Temporary auth state during tab session
 - One-time tokens that should die when the tab closes
 - Tab-specific state (active tab in dashboard)

Example

```
storage.js.html
2 <html lang="en" data-theme="light">
9 <body>
11 <div class="container">
20 <div class="card">
22 <h2>Checkout / Wizard (uses sessionStorage) / h2>
23 <div id="step-1" class="step active">
24 <h3>Step 1: Personal Info</h3>
25 <div class="input-group">
26 <input type="text" placeholder="Full name">
27 <input type="email" placeholder="Email">
28 </div>
29 <button class="next-btn" data-step="2">Next </button>
30 </div>
31
32 <div id="step-2" class="step">
33 <h3>Step 2: Shipping</h3>
34 <div class="input-group">
35 <input type="text" placeholder="Address line 1">
36 <input type="text" placeholder="City">
37 </div>
38 <div class="button-group">
39 <button class="back-btn" data-step="1">Back</button>
40 <button class="next-btn" data-step="3">Next </button>
41 </div>
42 </div>
43
44 <div id="step-3" class="step">
45 <h3>Step 3: Payment</h3>
46 <div class="input-group">
47 <input type="text" placeholder="Card number" maxleng=
48 <input type="text" placeholder="MM/YY" maxleng="
49 </div>
50 <div class="button-group">
51 <button class="back-btn" data-step="2">Back</button>
52 <button id="complete-order">Complete Order</button>
53 </div>
54 </div>
55 </div>
56 </body>
57 </html>
```

```
storage.js.css
63 .button-group {
64   display: flex;
65   justify-content: space-between;
66 }
67
68 .next-btn, #complete-order, #login-
69   background: #2563eb;
70   color: white;
71   padding: 10px 20px;
72   border-radius: 5px;
73 }
74
75 .back-btn {
76   background: #64748b;
77   color: white;
78   padding: 10px 20px;
79   border-radius: 5px;
80 }
81
82 #logout-btn {
83   background: #dc2626;
84   color: white;
85   padding: 10px 20px;
86   border-radius: 5px;
87 }
88
89 .hidden { display: none; }
90
91 .success { color: #10b981; font-weight: bold; }
92
93 pre {
94   background: #f1f5f9;
95   padding: 12px;
96   border-radius: 6px;
97   overflow-x: auto;
98   font-size: 0.9rem;
99 }
100
101 [data-theme="dark"] pre { background-color: #1e2430; color: #e0e0e0; }
```

```
storage.js.js
2 // 1. Theme switching (localStorage)
3 function setTheme(theme) {
4   document.documentElement.setAttribute("data-theme", theme);
5   localStorage.setItem("theme", theme);
6 }
7
8 const savedTheme = localStorage.getItem("theme");
9 setTheme(savedTheme);
10
11 document.querySelectorAll("[data-theme]").forEach((el) => {
12   el.addEventListener("click", () => {
13     setTheme(btn.getAttribute("data-theme"));
14   });
15 });
16
17 document.getElementById("reset-theme").addEventListener("click", () => {
18   localStorage.removeItem("theme");
19   setTheme("light");
20 });
21
22 // 2. Wizard / multi-step form (sessionStorage)
23 const stepDisplay = document.getElementById("step-display");
24
25 function showStep(n) {
26   document.querySelectorAll(".step").forEach((el) => {
27     el.classList.remove("active");
28     document.getElementById(`step-${n}`).classList.add("active");
29     stepDisplay.textContent = n;
30     sessionStorage.setItem("currentStep", n);
31   });
32 }
33
34 function loadStep() {
35   const saved = parseInt(sessionStorage.getItem("currentStep"));
36   showStep(saved);
37 }
38
39 document.querySelectorAll(".next-btn").forEach((btn) => {
40   btn.addEventListener("click", () => {
41     const currentStep = parseInt(sessionStorage.getItem("currentStep"));
42     const nextStep = currentStep + 1;
43     showStep(nextStep);
44   });
45 });
```



Security:

- **Both are plain text & accessible to any JS on the page** → XSS = game over (attacker can read tokens, preferences, etc.)
- **Never store JWTs / auth tokens directly in localStorage** if high security is needed → Prefer HttpOnly + Secure cookies or short-lived tokens + refresh mechanism
- **Use when convenience > perfect security** (themes, non-sensitive carts)
- **For sensitive data:** encrypt before storing (crypto.subtle) or avoid client storage

Summary:

- DOM selection → querySelector/All is the most flexible & modern default
- getElementById wins on speed for IDs
- Fetch → clean, Promise-based HTTP → use async/await + error handling
- Storage → localStorage = persistent, sessionStorage = tab-scoped
- Both vulnerable to XSS → never trust for secrets